Inteligência Artificial

Iago Gomes
iagogomes@ufscar.br

Roteiro

1. Descrição do Problema
2. Modelagem do mundo
3. Framework e API Mínima
4. Implementação de um Agente Aleatório
5. Implementação de um Agente Reativo Simples e de um Agente Reativo com Estados
6. Entrega e Relatório

1. Descrição do Problema

O Mundo do Wumpus

- O **Mundo do Wumpus** descreve a narrativa de um **herói** que **explora** um mundo **inicialmente desconhecido** em busca do **ouro**;
- Nesse mundo, o **Wumpus** e os **poços** representam **armadilhas fatais** para o herói;
- Entretanto, o herói possui recursos **sensoriais** para identificar a proximidade de uma armadilha fatal e quando encontrou o ouro;
- O **objetivo final** do jogo é encontrar o **ouro**;
- Mas, o **herói** também consegue **contra-atacar** e derrotar o **Wumpus**;
  - derrotar o **Wumpus** não acaba com o jogo, apenas adiciona um bônus na pontuação final;
  - as únicas condições de término são: **pegar o ouro**, **cair em um poço**, **ou ser derrotado pelo Wumpus**.

O Mundo do Wumpus: Mario Bros Version

- Nesse versão do jogo, os personagens são:
  - **Mário** como o **herói**;
  - **Bowser** como **Wumpus**;
  - **Princesa Peach** como **ouro**;

- Assim, o objetivo do **Mário** é resgatar a **Princesa Peach;**
- **Condições de sucesso e fracasso:**

SUCESSO

Alcançar a princesa e executar RESCUE.

FRACASSO

- cair em um poço;
- entrar na célula de Bowser vivo;

2. Modelagem do mundo

O Mundo

- O Mundo do Wumpus é descrito em termos de **estados**, **perceptos**, **ações** e **regras do ambiente**.

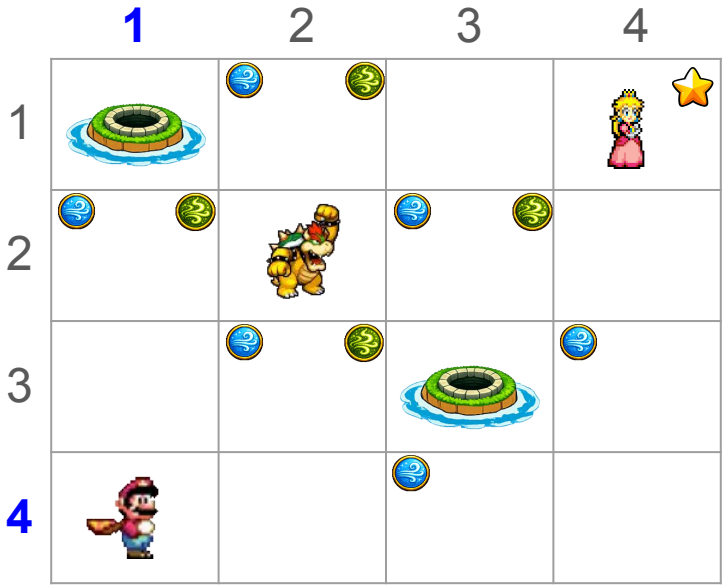
O Mundo

- O mundo é descrito como um grid **NxN**, em que **N** é o número de linhas e colunas;

	1	2	3	4
1				
2				
3				
4				

O Mundo

- Todos os elementos são posicionados no grid **aleatoriamente**, exceto **o Mário que sempre começa no (N, 1)**.



Perceptos e elementos do mundo

Ícone	Item	Descrição	Percepção	Símbolo
	Poços	Perigo fatal	Geram **breeze** nas células adjacentes	
	Bowser	Perigo fatal enquanto vivo	Gera **stink** nas adjacências.	
	Princesa	Objetivo	gera glitter na célula atual	
-	Grito	-	scream indica que Bowser foi eliminado por SHOOT.	-

Ações relevantes

Ícone	Ação	Percepção
-	MOVE_*	Move Mario e atualiza a direção atual.
	AIM_*	Muda apenas a direção de ataque.
	SHOOT	Usa a fireball na direção atual.
-	RESCUE	Resgata a princesa se ela estiver na célula.
-	WAIT	Não altera posição; útil para depuração e políticas simples.

3. Framework e API Mínima

API do Jogo

- A API do jogo é simples; isso permite focar nas decisões de projeto do agente.

Estrutura do Projeto

```
mario_wumpus_pygame/
    main.py
    REAME.md
    requirements.txt
    maps/
        fixed_4x4.json
    scripts/
        play_manual.py
        run_agent.py
    mario_wumpus/
        __init__.py
        app.py
        config.py
        core/
            actions.py
            env.py
            generator.py
            models.py
        agents/
            base.py
            greedy.py
            manual.py
            random_agent.py
        render/
            assets.py
            renderer.py
            ui.py
        assets/
            …
```

executar código

instalar dependências

arquivos com a implementação do jogo. [evite modificar]

implementação dos agentes

Classes Importantes

Classe	Descrição	Localização
BaseAgent	Classe Abstrata para o agente; expõe act(...) e, opcionalmente, reset().	./mario_wumpus/mario_wumpus/agents/base.py
Percept	Agrupa sinais locais, posição, direção, visitados e recursos.	./mario_wumpus/mario_wumpus/core/models.py
Action	Enum de ações disponíveis no ambiente.	./mario_wumpus/mario_wumpus/core/actions.py
WumpusEnv	Mantém o estado real do mundo e aplica env.step(action).	./mario_wumpus/mario_wumpus/core/env.py

Template Mínimo para um Agente

```python
from mario_wumpus.agentes.base import BaseAgent
from mario_wumpus.core.actions import Action

class MeuAgente (BaseAgent):
    def __init__(self, seed: int | None = None):
        pass

    def reset(self):
        pass

    def act(self, percept, legal_actions):
        return Action.WAIT
```

O framework - onde colocar e como executar

1) **Passo 1**: criar o arquivo
   a) Salvar o agente em **mario_wumpus/agents/**.
   b) Exemplos: simple_reactive_agent.py e state_reactive_agent.py.
2) **Passo 2** [Opcional]: testar rápido
   a) Editar **scripts/run_agent.py** e importar o agente desejado.
   b) Instanciar o agente e executar o episódio headless.
3) **Passo 3**: demonstrar no app
   a) Trocar o import/instância do agente automático em **mario_wumpus/app.py**.
      i) linha 120
   b) Executar **main.py** para visualizar o comportamento no tabuleiro.
      i) $ python3 main.py

4. Implementação de um Agente Aleatório

Implementação de um Agente Aleatório

- Implementação de um agente baseline bem simples.
- Estratégia:
  - se houver brilho, tenta `RESCUE`;
  - caso contrário, escolhe aleatoriamente entre mover, esperar ou atirar.

Implementação de um Agente Aleatório

```
1  class RandomAgent(BaseAgent):
2      def __init__(self, seed: int | None = None):
3        self.rng = random.Random(seed)
4        print("Agente Aleatório!")
5
6    def act(self, percept: Percept, legal_actions: list[Action]) -> Action:
7        if percept.glitter:
8            return Action.RESCUE
9        options = [
10           Action.MOVE_UP,
11           Action.MOVE_RIGHT,
12           Action.MOVE_DOWN,
13           Action.MOVE_LEFT,
14           Action.WAIT,
15       ]
16       if percept.has_fireball:
17           options.append(Action.SHOOT)
18       return self.rng.choice(options)
```

5. Implementação de um Agente Reativo Simples e de um Agente Reativo com Estados

Exercício A

- Descreva o PEAS e as características do ambiente do **Mundo Wumpus - Mario Bros Version**.

Exercício B

- Implemente um Agente Reativo Simples para resolver o **Mundo do Wumpus - Mario Bros Version**;
  - Um agente reativo simples deve decidir predominantemente a partir da percepção atual;
- **Dica**:
  - Registre o máximo de situações que podem ocorrer no jogo, e crie regras simples a partir delas;
  - Lembrem-se: **Reativo Simpes = Condição -> ação**.

Exercício C

- Implemente um Agente Reativo com Estados (Baseado em Modelo) para resolver o **Mundo do Wumpus - Mario Bros Version**;
- Dica:
  - Um modelo pode ser o mapa, um autômato, ambos (mapa + autômato), sistema de equações, etc.
  - A implementação pode partir de regras (condição -> ação) mais complexas, já que agora o agente tem disponível estados e um modelo;

6. Entrega e Relatório

Entrega e checklist final

- Entregas:
  - Arquivo do agente reativo simples funcionando.
  - Arquivo do agente reativo com estados funcionando.
  - Relatório curto em PDF.
- Condições do trabalho:
  - Trabalho individual.
  - Prazo de 2 semanas a partir da divulgação do trabalho no Classroom.
  - Entrega pelo Google Classroom.
  - Código deve funcionar no framework fornecido.
- Critérios observáveis:
  - Correção funcional dos agentes.
  - Coerência entre teoria e implementação.
  - Qualidade da modelagem explicada no relatório.
  - Clareza do código e das justificativas.

Relatório

Seção	O que descrever
1. Identificação	Nome, RA e data
2. Visão Geral do Problema	Objetivo do Agente no Mundo do Wumpus
3. Modelagem	Exercício A - PEAS + características do ambiente com justificativa curta
4. Agente Reativo Simples	Exercício B - Regras adotadas, perceptos usados e limitações
5. Agente com estados	Exercício C - Estados internos, transições ou memória usada, decisões, e limitações
6. Testes	Como testou e quais diferenças observou entre os agentes
7. Declaração de uso de IA	Declarar se usou e como usou IA no desenvolvimento do trabalho
8. Melhorias	Limitações, extensões futuras e feedback opcional do framework e projeto (ajudará as próximas turmas)